

1. Tell me about your current project

Situation:

In my current project, I am working as a Data Engineer on a public-sector analytics project for the Texas Veterans Commission. The main objective of the project is to bring data from multiple sources into a centralized Azure data platform and provide reliable analytics for veteran services. We receive data from on-premises SQL Server databases, CSV files, and REST APIs. The source systems have different structures and different refresh patterns, so one of the main challenges was creating a consistent and reliable data pipeline.

Task:

My responsibility is mainly around data ingestion, transformation, data quality, and preparing the data for analytics. I work with Azure Data Factory, ADLS Gen2, Databricks, PySpark, SQL, Synapse, Delta Lake, and Power BI. We needed to make the pipelines scalable, incremental, auditable, and easy for the reporting team to consume.

Action:

We implemented a Medallion architecture with Bronze, Silver, and Gold layers. ADF is used for orchestration and ingestion. Raw data first lands in ADLS Bronze. Databricks and PySpark are then used to clean, standardize, deduplicate, and transform the data into Silver. From Silver, we create business-ready Gold datasets using dimensional modeling and business rules. We use incremental loading with watermarks rather than processing the complete dataset every time. We also implemented validation and quarantine logic for invalid records. For example, we validate veteran IDs, dates, duplicates, and required fields. We maintain audit information so we know how many records were received, processed, rejected, and loaded.

Result:

This architecture helped us reduce unnecessary processing and made the pipelines more reliable. In one optimization, a Gold-layer query that was taking several minutes was reduced to a few seconds by optimizing the Delta tables and using appropriate partitioning/Z-Ordering. The final datasets are consumed by Power BI and reporting solutions for claims, healthcare utilization, and veteran service performance.

Interview closing:

"So overall, my current role is a combination of Azure data engineering, SQL, PySpark, data quality, dimensional modeling, and preparing trusted data for reporting."

2. You have worked on Snowflake, ADLS, and SQL. How much would you rate yourself out of 10?

I would rate myself differently depending on the technology because my hands-on experience is not exactly the same across all three.

For **SQL**, I would rate myself around **8.5 to 9 out of 10**. I have strong experience with SQL Server, including joins, CTEs, subqueries, window functions, stored procedures, data validation, performance tuning, and ETL-related SQL. I use SQL regularly for source analysis, transformation, reconciliation, and validation.

For **ADLS**, I would rate myself around **8 out of 10**. I have hands-on experience with ADLS Gen2 as part of Azure data pipelines. I have worked with Bronze, Silver, and Gold storage layers, file ingestion, folder structures, incremental processing, metadata, and integration with ADF and Databricks.

For **Snowflake**, I would rate myself around **6.5 to 7 out of 10**. I have strong knowledge of Snowflake architecture and have been working extensively on Snowflake development and interview scenarios, including stages, file formats, COPY INTO, Snowpipe, Streams, Tasks, Dynamic Tables, Time Travel, micro-partitions, caching, security, and SQL. My strongest production hands-on experience has been with Azure technologies, while Snowflake is an area I have been actively building deeper practical experience in.

I prefer giving an honest rating rather than saying 10 out of 10 because there is always more to learn.

Good closing sentence:

"If I get an issue in any of these technologies, I am comfortable troubleshooting it, researching the root cause, and implementing the solution."

3. What architecture did you use in your current project?

Situation:

Our source data was coming from multiple systems, including SQL Server, CSV files, and APIs. The data was not standardized, and reporting directly from source systems would have created performance and data-quality problems.

Task:

We needed an architecture that could handle raw data, transformation, validation, historical data, and business-ready datasets separately.

Action:

We implemented a **Medallion architecture** using Bronze, Silver, and Gold layers.

The **Bronze layer** stores the data in its raw or minimally changed form. This gives us a historical copy of what came from the source.

The **Silver layer** contains cleaned and standardized data. Here we handle data types, duplicates, nulls, invalid records, standardization, joins, and business-independent transformations.

The **Gold layer** contains business-ready datasets. We apply business rules, create dimensional models, and prepare data for Power BI and reporting.

ADF handles orchestration and ingestion. ADLS is used for storage. Databricks and PySpark handle large-scale transformation. Synapse is used for analytical access, and Power BI consumes the curated data.

We also implemented incremental loading using watermarks, audit logging, validation, and quarantine processes.

Result:

This architecture separated ingestion from transformation and reporting. It made the system easier to maintain, troubleshoot, and scale. If there is a problem in the Gold layer, we can trace the data back through Silver and Bronze to the original source.

Simple interview summary:

"ADF orchestrates, ADLS stores, Databricks transforms, Synapse serves analytical data, and Power BI consumes the business-ready data."

4. In Medallion architecture, what is the use of the Bronze layer?

Situation:

In our project, source data was coming from SQL Server, CSV files, and APIs. We needed a place to store the original data before applying business transformations.

Task:

The purpose of Bronze was to preserve the source data and create a reliable starting point for downstream processing.

Action:

We load the source data into the Bronze layer with minimal transformation. We generally preserve the original structure as much as practical and add technical metadata such as source file name, load timestamp, and processing information.

For example, if a CSV contains 100,000 records, we first land that data in Bronze. We don't immediately apply complex business rules. This gives us traceability.

Suppose the Silver transformation accidentally removes some records. We can compare Silver against Bronze and identify what happened. If the source system sends corrected data later, we can also reprocess from Bronze instead of asking the source team to resend everything.

Bronze also helps with auditing, replay, troubleshooting, and historical traceability.

Result:

Bronze gives us a trusted raw landing area and protects us from losing the original source data during transformation.

Simple explanation:

"I think of Bronze as the raw backup and historical landing zone. Silver is where we clean the data, and Gold is where we make it business-ready."

5. Why did you use ELT and not ETL?

Situation:

Our source systems generate a significant amount of data, and we use cloud storage and scalable compute resources.

Task:

We wanted to avoid doing all transformations before storing the data because that can create unnecessary processing and reduce flexibility.

Action:

With **ETL**, we extract the data, transform it outside the target system, and then load the transformed data.

With **ELT**, we extract the data, load it into the cloud storage/platform, and then transform it using scalable processing engines.

In our architecture, we first ingest the data into ADLS Bronze. Then Databricks/PySpark performs transformations, and the curated data moves through Silver and Gold.

ELT gives us the ability to preserve raw data and apply different transformations later. For example, if the business changes a calculation, we don't necessarily need to go back to the source system. We can reprocess the data already available in the Bronze layer.

Result:

This approach provides better scalability, traceability, and flexibility for analytics workloads.

Important interview point:

"ETL is not wrong. I choose between ETL and ELT based on the architecture, volume, transformation requirements, and capabilities of the target platform."

6. Have you worked on Snowflake Dynamic Tables?

I have worked extensively with the concept and implementation patterns around Snowflake Dynamic Tables, and I understand how they fit into modern Snowflake pipelines. My strongest production experience has been with Azure data engineering, so I would describe my Snowflake Dynamic Table experience as **hands-on development and project-level practical knowledge rather than many years of production ownership.**

A Dynamic Table is useful when I want Snowflake to automatically maintain the result of a SQL transformation. Instead of manually creating a pipeline with a Stream, Task, and multiple procedures for every transformation, I can define the desired result using SQL and specify a target freshness.

For example:

```
CREATE DYNAMIC TABLE customer_summary
TARGET_LAG = '10 minutes'
WAREHOUSE = COMPUTE_WH
AS
SELECT
    customer_id,
    COUNT(*) AS order_count,
    SUM(order_amount) AS total_amount
FROM orders
GROUP BY customer_id;
```

Here, the Dynamic Table automatically refreshes based on its target lag and upstream data changes.

The important distinction I explain in interviews is:

- **Stream** tells me what changed.
- **Task** schedules or triggers work.
- **Stored procedure** contains procedural/business logic.
- **Dynamic Table** maintains a transformed table based on a SQL definition.

If the requirement is primarily declarative transformation and freshness, Dynamic Tables are useful. If I need custom procedural logic or complex orchestration, I would consider Streams, Tasks, and stored procedures.

7. Suppose there is PHI/PII information in production data and you want to move it to a lower environment. How will you handle that?

Situation:

Suppose production contains sensitive information such as name, email, phone number, address, Social Security Number, or healthcare-related PHI.

Task:

The goal is to provide developers and testers with realistic data without exposing actual sensitive production information.

Action:

I would not simply copy the production table into DEV or SIT.

First, I identify the sensitive columns and classify them as PII or PHI. Then I determine whether each column needs masking, tokenization, anonymization, or removal.

For example:

```
John Smith      -> Customer_10001
john@email.com  -> user10001@test.com
123-45-6789    -> XXX-XX-6789
```

For identifiers used for joins, I would use **consistent masking**. This is important because if Customer ID is masked differently in different tables, relationships will break.

For example:

```
Customer 1001 -> HASH_VALUE_ABC
Customer 1001 -> HASH_VALUE_ABC
```

The same original customer should receive the same masked value across related tables.

I would also restrict who can access the lower environment and make sure sensitive data is not exposed through logs, files, temporary tables, or debugging outputs.

Result:

The testing team gets realistic data structures and relationships without receiving the actual sensitive production values.

Key sentence:

"My approach is to preserve the analytical usefulness of the data while removing the ability to identify the real individual."

8. What security will you implement for that kind of data?

I would use multiple layers of security rather than depending on one feature.

At the **Snowflake level**, I would implement role-based access control using roles and privileges. Users should only receive the permissions required for their job.

For sensitive columns, I can use **Dynamic Data Masking**. For example, an analyst may see only part of an email address while an authorized security role can see the full value.

I can also use **Row Access Policies** when different users should see different rows.

For sensitive data, I would also consider **tagging and classification**, so sensitive columns can be identified and governed consistently.

At the storage level, I would restrict access to ADLS using Azure identity and access controls.

I would also implement encryption, auditing, monitoring, and secure service identities.

For lower environments, I would preferably use masked or anonymized data rather than real production PII/PHI.

Interview summary:

"I would use least privilege, role-based access control, masking, row-level security where required, classification/tagging, encryption, auditing, and non-production data masking."

9. Is there any Snowflake configuration or table-level feature you can use for this?

Yes. Snowflake provides several features that can help protect sensitive data.

One important feature is **Dynamic Data Masking**. A masking policy can determine what a user sees based on their role.

For example:

```
CREATE MASKING POLICY mask_email
AS (val STRING)
RETURNS STRING ->
CASE
    WHEN CURRENT_ROLE() IN ('PII_ADMIN')
    THEN val
    ELSE '*****'
END;
```

Then I can apply the policy to a column.

Another feature is **Row Access Policy**, which controls which rows a user can see.

Snowflake also supports **tags and classification** for governance. Sensitive columns can be tagged as PII or other sensitive categories.

For lower environments, I would still prefer to create masked/anonymized data rather than relying only on a policy.

The important point is that security should be implemented at multiple layers.

Interview answer:

“Yes. Snowflake provides masking policies, row access policies, tags/classification, role-based access control, and auditing capabilities. For PII/PHI, I would combine these features rather than depending on one control.”

10. SQL Server to Snowflake migration: Table A and Table B have the same columns but no primary key. How will you validate missing or mismatching records?

Situation:

Suppose Table A contains the migrated data and Table B contains the SIT validation data. Both have the same columns, but there is no primary key.

Task:

I need to identify: 1. records present in A but not B 2. records present in B but not A 3. records where values are different

Action:

First, I would compare record counts.

```
SELECT COUNT(*) FROM table_a;

SELECT COUNT(*) FROM table_b;
```

Then, if there is a logical business key, I would use that key even if there is no physical primary-key constraint.

For example:

```
SELECT
    COALESCE(a.customer_id, b.customer_id) AS customer_id,
```



```

a.name AS a_name,
b.name AS b_name,
a.salary AS a_salary,
b.salary AS b_salary
FROM table_a a
FULL OUTER JOIN table_b b
  ON a.customer_id = b.customer_id
WHERE a.customer_id IS NULL
  OR b.customer_id IS NULL
  OR NVL(a.name, '') <> NVL(b.name, '')
  OR NVL(a.salary, 0) <> NVL(b.salary, 0);

```

The `FULL OUTER JOIN` is important because it returns records from both sides.

If there is genuinely no unique business key, I would use a **row hash**.

For example:

```

MD5(
  CONCAT_WS(' ',
    COALESCE(customer_id, ''),
    COALESCE(name, ''),
    COALESCE(salary, '')
  )
)

```

Then I compare the hashes between the two datasets.

I would also compare aggregate checks such as counts, sums, and distinct counts.

Result:

This gives me record-level and dataset-level validation rather than relying only on row counts.

11. What is the difference between EXCEPT and MINUS?

Both are used to return rows that exist in one result set but not the other.

In Snowflake, **EXCEPT** and **MINUS** are supported as set operators and are effectively synonyms for this purpose.

For example:

```
SELECT customer_id
FROM table_a

EXCEPT

SELECT customer_id
FROM table_b;
```

This returns customer IDs that exist in Table A but not Table B.

You can also write:

```
SELECT customer_id
FROM table_a

MINUS

SELECT customer_id
FROM table_b;
```

The result is equivalent.

The important point is that the order matters.

```
A EXCEPT B
```

means records in A but not B.

While:

```
B EXCEPT A
```

means records in B but not A.

I normally use `EXCEPT` because it is more commonly understood across SQL platforms, but if the database supports `MINUS`, both can be used.

12. Suppose I loaded one file into ADLS Storage and want to load it into a Snowflake table. What approach will you follow?

Situation:

Suppose a CSV file is available in ADLS Gen2 and I need to load it into Snowflake.

Task:

I need to establish connectivity between Snowflake and ADLS and then load the file safely.

Action:

I would create an **external stage** in Snowflake that points to the ADLS location.

First, I would define the file format.

```
CREATE FILE FORMAT csv_format
TYPE = CSV
FIELD_DELIMITER = ','
SKIP_HEADER = 1
FIELD_OPTIONALLY_ENCLOSED_BY = '';
```

Then I would create an external stage pointing to ADLS.

Conceptually:

```
CREATE STAGE adls_stage
URL = 'azure://<storage-account>/<container>/<path>'
FILE_FORMAT = csv_format;
```

Authentication is configured using the appropriate Azure storage integration/security mechanism.

Then I can verify the files:

```
LIST @adls_stage;
```

Finally, I load the data:

```
COPY INTO target_table
FROM @adls_stage;
```

I would then validate:

```
SELECT COUNT(*)
FROM target_table;
```

and compare the result with the source file or source-system record count.

Interview summary:

"ADLS is the external storage, Snowflake external stage provides access to that storage, file format defines how Snowflake interprets the file, and COPY INTO loads the data into the Snowflake table."

13. After successful COPY INTO, I want to delete that file from ADLS. How will you do that?

I would normally separate the loading step and file cleanup step.

`COPY INTO` is responsible for loading the data. After I verify that the load was successful, I would use a separate process to archive or delete the source file.

For example, I could use Azure Data Factory or another orchestration process:

```
ADLS
  ↓
Snowflake COPY INTO
  ↓
Validate load
  ↓
If successful
  ↓
Archive/Delete file
```

I prefer archiving instead of immediately deleting in many production systems because it provides traceability and allows recovery.

For example:

```
landing/  
  file1.csv  
  
archive/  
  file1.csv
```

After a successful load, ADF can move the file from the landing folder to the archive folder.

If the business requirement specifically requires deletion, ADF can delete the file after successful validation.

The important point is that I should not delete the file before confirming the Snowflake load was successful.

14. Inside COPY INTO, is there any property for that?

There is a Snowflake option called **PURGE** that can be used with `COPY INTO`.

Example:

```
COPY INTO target_table  
FROM @adls_stage  
PURGE = TRUE;
```

The purpose of `PURGE = TRUE` is to remove successfully loaded files from the stage location after the copy operation.

However, I would be careful with this in production because deleting the source file immediately can affect recovery and auditing.

If the organization's process requires retaining files, I would use an archive process instead.

So my interview answer would be:

"Yes, Snowflake COPY INTO supports the `PURGE` option. With `PURGE = TRUE`, successfully loaded files can be removed from the stage location. But in production I would first confirm the retention and recovery requirements before using it."

15. In which case will you use Snowpipe?

Situation:

Suppose files arrive continuously in ADLS throughout the day. Instead of waiting for a scheduled batch process every few hours, the business wants the data available shortly after arrival.

Task:

We need near-real-time or continuous file ingestion.

Action:

This is where I would consider **Snowpipe**.

Snowpipe is designed for continuous or near-real-time loading of files into Snowflake. When new files arrive in cloud storage, an event notification mechanism can notify Snowflake, and Snowpipe loads the files automatically.

The architecture looks like:

```
Source
  ↓
ADLS
  ↓
New file arrives
  ↓
Cloud event notification
  ↓
Snowpipe
  ↓
Snowflake table
```

For a daily batch of 100 files, I might use `COPY INTO` through a scheduled process.

For files arriving continuously throughout the day, Snowpipe is more appropriate.

I would also configure monitoring and error handling so failed files can be identified and reprocessed.

Key difference:

"COPY INTO is generally something I explicitly execute, while Snowpipe is designed for continuous automated ingestion as new files arrive."

16. What is the difference between a Snowflake Task and a Stored Procedure?

A **Task** is primarily used for scheduling or triggering work.

A **Stored Procedure** is used to contain procedural logic.

For example, a Task can run every hour:

```
CREATE TASK daily_task
WAREHOUSE = COMPUTE_WH
SCHEDULE = 'USING CRON 0 * * * * UTC'
AS
CALL load_customer_data();
```

Here the Task controls **when** the work runs.

The Stored Procedure controls **what the work does**.

For example:

```
CREATE PROCEDURE load_customer_data()
RETURNS STRING
LANGUAGE SQL
AS
$$
BEGIN
    INSERT INTO customer_target
    SELECT *
    FROM customer_stage;

    RETURN 'Load completed';
END;
$$;
```

So I remember it this way:

Task = when/how to trigger

Stored Procedure = what logic to execute

They can also work together.

For example:

```
Task
↓
Stored Procedure
↓
Validation
↓
Transformation
↓
Target table
```

A Task can also execute SQL directly, but when I have complex procedural logic, a stored procedure can be useful.

17. What is Time Travel?

Situation:

Suppose someone accidentally deletes or updates data in Snowflake.

Task:

I need to recover or inspect the previous version of the data without having a traditional backup restored.

Action:

Snowflake Time Travel allows me to query historical data from a previous point in time within the configured retention period.

For example:

```
SELECT *
FROM customer
AT (TIMESTAMP => '2026-09-16 10:00:00');
```

Or using an offset:

```
SELECT *
FROM customer
AT (OFFSET => -60*5);
```

This lets me inspect the table as it existed at an earlier time.

If a table was accidentally dropped, depending on the situation and retention period, Time Travel and `UNDROP` can help recover it.

For example:

```
UNDROP TABLE customer;
```

Time Travel is useful for accidental deletes, incorrect updates, auditing, and recovery.

The important distinction is that Time Travel is not a replacement for a complete backup strategy. It provides historical access within the applicable retention period.

18. What is the difference between the architecture of SQL Server and Oracle versus Snowflake?

SQL Server and Oracle traditionally use a more tightly integrated database architecture where compute, storage, memory, and database processing are managed together within database infrastructure.

Snowflake uses a **cloud-native architecture** that separates **storage and compute**.

Conceptually:

SQL Server / Oracle

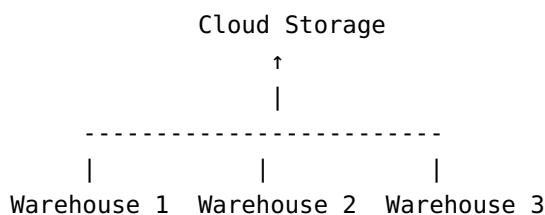
Database

├─ Storage

├─ Memory

└─ Compute

Snowflake:



Snowflake stores data in cloud storage and uses independent virtual warehouses for compute.

This means different workloads can use separate compute resources.

For example, reporting can use one warehouse while ETL uses another.

Another difference is that Snowflake manages many infrastructure-level responsibilities for the user, including automatic micro-partitioning and cloud infrastructure management.

Snowflake also uses a columnar storage format and metadata-driven micro-partitions.

SQL Server and Oracle can also be highly scalable, but their traditional architecture and operational model are different from Snowflake's cloud-native separation of storage and compute.

Interview summary:

"The main architectural difference I would highlight is that Snowflake separates storage and compute, while traditional SQL Server and Oracle deployments are more tightly coupled around database infrastructure."

19. How much experience do you have with Copilot / AI-assisted tools?

I have experience using AI-assisted tools mainly to improve development productivity, troubleshooting, documentation, and learning.

For example, when working on SQL or PySpark, I can use AI assistance to understand an error, generate an initial query, explain unfamiliar syntax, or suggest optimization approaches. But I don't treat AI-generated code as automatically correct.

My process is:

```
Business requirement
    ↓
AI assistance if useful
    ↓
Review generated code
    ↓
Test against sample data
    ↓
Validate results
    ↓
Deploy
```

For example, if an AI tool generates a SQL query using a window function, I still check whether the partitioning and ordering are correct and whether duplicate records could change the result.

I also use AI-assisted tools for documentation and explaining technical concepts.

My approach is that AI is a productivity tool, not a replacement for engineering judgment. I remain responsible for validating security, performance, correctness, and business logic.

20. SQL: Write a query to find the top 3 products by sales.

Assume the table is:

```
sales
-----
product_id
product_name
quantity
unit_price
```

I would write:

```
SELECT
    product_id,
    product_name,
    SUM(quantity * unit_price) AS total_sales
FROM sales
GROUP BY
    product_id,
    product_name
ORDER BY
    total_sales DESC
LIMIT 3;
```

Explanation

First:

```
SUM(quantity * unit_price)
```

calculates total sales for each product.

Then:

```
GROUP BY product_id, product_name
```

groups all transactions belonging to the same product.

Then:

```
ORDER BY total_sales DESC
```

sorts products from highest sales to lowest sales.

Finally:

```
LIMIT 3
```

returns only the top three products.

If the interviewer asks about ties, I would use `DENSE_RANK()` :

```
SELECT *
FROM (
    SELECT
        product_id,
        product_name,
        SUM(quantity * unit_price) AS total_sales,
        DENSE_RANK() OVER (
            ORDER BY SUM(quantity * unit_price) DESC
        ) AS sales_rank
    FROM sales
    GROUP BY product_id, product_name
)
WHERE sales_rank <= 3;
```

This version handles ties more explicitly.

21. SQL: Using employee salaries, find employees whose salary increased compared to the previous year.

Assume:

```
employee_salary
-----
employee_id
employee_name
year
salary
```

I would use the `LAG()` window function.

```
SELECT
    employee_id,
    employee_name,
    year,
    salary,
    LAG(salary) OVER (
        PARTITION BY employee_id
        ORDER BY year
    ) AS previous_salary
FROM employee_salary;
```

Then compare the current salary with the previous salary:

```
SELECT *
FROM (
    SELECT
        employee_id,
        employee_name,
        year,
        salary,
        LAG(salary) OVER (
            PARTITION BY employee_id
            ORDER BY year
        ) AS previous_salary
    FROM employee_salary
```

```
)  
WHERE salary > previous_salary;
```

Explanation

`PARTITION BY employee_id` means the comparison happens separately for each employee.

`ORDER BY year` makes sure we compare the current year with the previous year.

`LAG(salary)` retrieves the salary from the previous row.

For example:

Employee	Year	Salary	Previous Salary
101	2023	70000	NULL
101	2024	75000	70000
101	2025	73000	75000

The query returns 2024 because:

```
75000 > 70000
```

It does not return 2025 because:

```
73000 < 75000
```

This is a common example of why window functions are useful for year-over-year analysis.

22. Do you have work experience on Python?

Yes. I have used Python primarily for data engineering and data processing rather than traditional application development.

In my projects, Python is mainly used through **PySpark** for large-scale data transformation.

For example, I use Python/PySpark to read data, clean records, remove duplicates, apply transformations, join datasets, validate records, and write the results to Delta tables.

A simple example is:

```
df = spark.read.format("csv")  
    .option("header", "true")  
    .load("/bronze/customer.csv")
```

Then I can clean the data:

```
df_clean = df.dropDuplicates(["customer_id"])
```

I can filter invalid records:

```
df_valid = df_clean.filter(  
    df_clean.customer_id.isNotNull()  
)
```

Then write it to the Silver layer:

```
df_valid.write  
    .format("delta")  
    .mode("overwrite")  
    .save("/silver/customer")
```

I have also used Python for data validation and automation.

My main strength is using Python in combination with Spark and cloud data platforms for data engineering workloads.

23. Suppose you already returned a DataFrame and I want the last 5 records. How will you get them?

This depends on whether the DataFrame has a meaningful ordering column.

This is important because a DataFrame is not inherently guaranteed to have a meaningful "last" row unless I specify an ordering.

For example, if I have an `id` column and want the five highest IDs:

```
df.orderBy(  
    df.id.desc()  
) .limit(5)
```

If I want them displayed in ascending order afterward:

```
last_5 = df.orderBy(  
    df.id.desc()  
) .limit(5)  
  
last_5.orderBy("id").show()
```

Explanation

First:

```
orderBy(df.id.desc())
```

sorts the records from highest ID to lowest ID.

Then:

```
limit(5)
```

gets the five highest records.

If the interviewer says "last five based on date," I would use the date column:

```
df.orderBy(  
    df.transaction_date.desc()  
) .limit(5)
```

I would also clarify that I need a business column such as ID, timestamp, or date to define what "last" means.

24. Suppose I want to rename a column in PySpark. How will you do that?

The simplest method is `withColumnRenamed()`.

```
df = df.withColumnRenamed(  
    "old_column_name",  
    "new_column_name"  
)
```

For example:

```
df = df.withColumnRenamed(  
    "cust_id",  
    "customer_id"  
)
```

This changes the column name from `cust_id` to `customer_id`.

If I need to rename multiple columns:

```
df = df.withColumnRenamed("cust_id", "customer_id")  
      .withColumnRenamed("cust_name", "customer_name")
```

Another approach is using `select` with `alias`:

```
from pyspark.sql.functions import col  
  
df = df.select(  
    col("cust_id").alias("customer_id"),  
    col("cust_name").alias("customer_name")  
)
```

I normally use `withColumnRenamed()` when I only need to rename columns, and `select/alias` when I am already selecting or transforming columns.

25. I have two DataFrames and both have an ID column. I want to merge them into a third DataFrame. How will you do that?

First I would understand what "merge" means in the requirement.

If the interviewer means combine records based on the ID, I would use a **join**.

For example:

```
df3 = df1.join(
    df2,
    df1.id == df2.id,
    "inner"
)
```

This creates a third DataFrame containing records where the ID exists in both DataFrames.

If I want all records from both DataFrames:

```
df3 = df1.join(
    df2,
    df1.id == df2.id,
    "outer"
)
```

If I want all records from the first DataFrame and matching records from the second:

```
df3 = df1.join(
    df2,
    df1.id == df2.id,
    "left"
)
```

If both DataFrames contain a column with the same name, I may select the columns explicitly:

```
df3 = df1.alias("a").join(
    df2.alias("b"),
    col("a.id") == col("b.id"),
    "inner"
)
```

```
.select(  
  col("a.id"),  
  col("a.name"),  
  col("b.salary")  
)
```

Explanation

`alias()` gives each DataFrame a temporary name.

```
df1.alias("a")  
df2.alias("b")
```

This makes it clear which `id`, `name`, or other column I am referring to.

The join condition:

```
col("a.id") == col("b.id")
```

matches records where both IDs are equal.

The result is stored in:

```
df3
```

So my interview answer would be:

"If I need to combine two DataFrames based on a common ID, I use a join. The join type depends on the requirement: inner for matching records, left for all records from the first DataFrame, and outer when I need records from both sides."

QUICK INTERVIEW CHEAT SHEET

Question	Main point to remember
Current project	Azure Data Engineering + Medallion + ADF + ADLS + Databricks + PySpark + Synapse
Rating	SQL 8.5–9, ADLS 8, Snowflake 6.5–7
Architecture	Medallion: Bronze → Silver → Gold

Question	Main point to remember
Bronze	Raw/history/traceability
ELT	Load first, transform using scalable cloud compute
Dynamic Tables	Declarative automatically refreshed tables
PHI/PII	Mask/anonymize before lower environment
Security	RBAC + masking + row access + tags + encryption + audit
Snowflake protection	Masking policies + row access policies
Table validation	Count + FULL OUTER JOIN + EXCEPT + hashes
EXCEPT/MINUS	Same set-difference concept in Snowflake
ADLS → Snowflake	External stage → file format → COPY INTO
Delete file	Archive/delete after successful validation
COPY property	<code>PURGE = TRUE</code>
Snowpipe	Continuous/near-real-time file ingestion
Task	Scheduling/orchestration
Stored Procedure	Procedural/business logic
Time Travel	Query/recover historical data
Snowflake architecture	Storage and compute separation
Copilot/AI	Productivity tool, always validate output
Top 3 products	<code>GROUP BY + SUM + ORDER BY + LIMIT</code>
Salary increase	<code>LAG()</code>
Python	PySpark/data engineering
Last 5 DataFrame rows	<code>orderBy(desc()).limit(5)</code>
Rename column	<code>withColumnRenamed()</code>
Merge DataFrames	<code>join()</code>

Strong closing statement for the interview

If the interviewer asks several of these questions one after another, keep your answers connected to your actual experience:

"My core strength is data engineering using SQL and Azure technologies. In my current project I work with ADF, ADLS, Databricks, PySpark, Synapse and Delta Lake, using a Medallion architecture. I also have strong SQL experience and have been building deeper hands-on knowledge in Snowflake, particularly around COPY INTO, stages, Snowpipe, Tasks, Streams, Dynamic Tables, Time Travel and Snowflake security. When I work with any technology, I focus not only on writing the code but also on data quality, performance, security, monitoring and making sure the solution meets the business requirement."